Réécrire au caractère : un langage de messagerie à transclusion 2D

Pierre Gradit

Version de travail – 22 octobre 2026

Résumé

Nous proposons un langage de messagerie dont chaque message réécrit, au caractère, un document affiché dans un panneau de résultat. Depuis `ed`, les outils de gestion de configuration manipulent le texte à la ligne ; notre langage descend au caractère par deux constructions : la coupure `<m:i>`, qui compte pour un caractère dans le flux, et la transclusion `<X>`, bloc dont l'intérieur reste adressable. Distribués, les messages sont nommés par leur hash SHA-256, ce qui fait de l'historique un DAG de Merkle. Chaque message définit une règle de réécriture positionnée `<X>` $\to$ X ; l'opérateur `@>` en extrait la relation décontextualisée, utile pour la preuve, et l'opérateur `@>>` la replace dans son contexte à la manière d'un Journal officiel. Le modèle, précurseur du Web3 en 2018, trouve une seconde jeunesse dans l'outillage des collections d'agents.

1 Introduction

La messagerie est devenue l'interface de travail par défaut, pour les humains comme pour les agents. Son agencement est stable : interlocuteurs et groupes à gauche, flux des messages au centre. Nous y ajoutons un panneau de résultat à droite, où s'affiche le document que les messages construisent.

Dans ce cadre, un message n'est plus seulement une prise de parole. C'est aussi une opération sur le document : il cite, coupe, insère et recouvre. Le flux de la conversation devient l'historique des versions, et le panneau droit en est le rendu courant.

Ce modèle a été formulé en 2018, dans le contexte du Web3, comme un registre de réécritures en ajout seul. Il prend une seconde jeunesse avec les collections d'agents, qui ont besoin d'un langage d'édition compact, vérifiable et adressable au caractère. Il s'appuie sur la théorie de la réécriture, en particulier de graphes [7], pour fonder ses propriétés.

Nos contributions sont les suivantes :

1. un langage de messagerie qui réécrit un document au caractère, par coupures et transclusions ;
2. une transclusion en deux dimensions, qui distingue un intérieur et un extérieur ;
3. une lecture de chaque message comme règle de réécriture positionnée, avec deux opérateurs, `@>` et `@>>` ;
4. un pont entre chaîne de hashes et légistique, par l'analogie avec le Journal officiel.

2 De `ed` à Git : la granularité de la ligne

Tous les systèmes de gestion de configuration qui reposent sur un programme de réécriture descendent de `ed`, le premier éditeur de texte d'UNIX. `ed` opère par lignes : la ligne est donc l'unité gérée de CVS jusqu'à Git.

La filiation est directe. L'algorithme de Hunt et McIlroy [10] produit, avec `diff -e`, un script `ed`. RCS stocke les versions sous forme de deltas inverses de ce type, et CVS s'appuie sur RCS. Les outils de fusion héritent de la même unité.

Git nuance le tableau sans le contredire. Il stocke des instantanés, et ses deltas de compression dans les *packfiles* sont des instructions binaires de copie et d'insertion à l'octet, proches de VCDIFF [11]. Mais la présentation du diff, la détection des conflits et la fusion restent à la ligne. La granularité ligne est celle de l'interaction et de la fusion, pas celle du stockage.

Notre langage porte au caractère ce qui restait à la ligne : l'adresse d'une modification, la citation de ce qu'elle remplace, et donc le lieu du conflit.

3 Le langage : coupures et liste numérotée

Le modèle élémentaire est la liste numérotée, comme en Markdown : chaque message reçoit un numéro, et un message ultérieur peut y faire référence. Une référence est une **coupure** `<m:i>`, notée comme une balise XML.

- En ouverture, `<m:i>` conserve le préfixe du message *m* jusqu'au caractère *i* inclus.
- En fermeture, `<m:i>` conserve le suffixe du message *m* qui suit le caractère *i*.
- Une coupure compte pour **un seul caractère** dans le message qui la contient.

Une coupure est donc une position entre deux caractères, et non un caractère. Son rôle, préfixe ou suffixe, est fixé par sa place dans le message.

3.1 Exemple à trois messages

```
1. Ceci est un paragraphe.
2. <1:12>texte réécrit<1:22>
3. <2:7>avec un repentir non trivial. Pour continuer :<1:23>
```

Message	Rendu	Longueur brute
1	Ceci est un paragraphe.	23
2	Ceci est un texte réécrit.	15
3	Ceci est un texte avec un repentir non trivial. Pour continuer :	48

Table 1: Rendus de l'exemple à trois messages.

Le message 2 conserve « Ceci est un » (les 12 premiers caractères de 1), insère « texte réécrit », puis reprend le suffixe de 1 après le caractère 22, soit le point final.

Le message 3 est plus instructif. Dans la forme brute de 2, la coupure `<1:12>` occupe la position 1, puis viennent t(2), e(3), x(4), t(5), e(6) et l'espace (7). `<2:7>` produit donc « Ceci est un texte ». La coupure de fermeture `<1:23>` reprend le suffixe vide de 1, ce qui supprime le point.

Le message 3 cite **deux ancêtres**, 2 et 1. Un message peut ainsi fusionner plusieurs versions, et l'historique forme un graphe orienté acyclique, non une chaîne. Comme une référence ne pointe que vers un numéro antérieur, l'expansion termine toujours.

4 Transclusion 2D et distribution

Distribuée, la messagerie remplace les numéros de messages par leurs hashes. Chaque réécriture véhicule alors l'entropie de ses références internes : le hash d'un message engage tout l'historique qu'il touche.

4.1 Séquençage et hachage

Le message brut est sérialisé en UTF-8 : le texte tel quel, une coupure sous la forme `<hash:n>`, une transclusion sous la forme `<hash>`, avec le hash complet en hexadécimal. On applique SHA-256 à cette chaîne. Les hashes sont présentés sous la forme abrégée `749a...986b` (quatre premiers et quatre derniers caractères).

	Hash	Message brut
A	`749a...986b`	Ceci est un paragraphe.
B	`0631...7a3c`	`<749a...986b:12>`texte réécrit`<749a...986b:22>`
C	`70fd...3c77`	`<0631...7a3c:7>`avec un repentir non trivial. Pour continuer :`<749a...986b:23>`

Table 2: L'exemple à trois messages, nommé par hashes.

Une coupure compte pour un caractère, qu'elle s'écrive `<1:12>` ou avec 64 caractères hexadécimaux. Les offsets sont donc invariants : le passage des numéros aux hashes est un pur renommage, une α-conversion. C dépend de A par deux chemins, directement et à travers B ; l'historique est un DAG de Merkle.

4.2 La transclusion

La transclusion `<X>` insère un bloc. Contrairement à la coupure, elle compte pour la longueur de ce qu'elle marque, et son intérieur est adressable.

```
4. <A:0><A>, et aussi<A:4>
```

Ici `<A>` compte pour 23. Le message 4 remplace « Ceci » par la copie entière de A suivie de « , et aussi ». Son rendu est « Ceci est un paragraphe., et aussi est un paragraphe. ».

Un message peut aussi ne rien insérer. Le message 5 supprime le point final de A dans sa copie dans 4 :

```
5. <4:23><4:24>
```

Son rendu est « Ceci est un paragraphe, et aussi est un paragraphe. » : seule la copie perd son point, A reste intact. Le message 5 porte la règle « . » $\rightarrow$ « », de membre droit vide, et son hash est `d807...29f7`.

4.3 Intérieur et extérieur

Xanadu [15] pensait la transclusion en trois dimensions. Notre modèle la pense en deux : la position du caractère et la couche. Cette réduction a un avantage décisif : elle fait exister un intérieur et un extérieur. Sans intérieur, pas de sécurité.

Une réécriture sur 4 qui vise l'intérieur de la copie de A reste **au-dessus** de la copie :

```
6. <4:13>chapitre<4:23>
```

Rendu : « Ceci est un chapitre., et aussi est un paragraphe. ». Seule la copie est touchée ; le second « paragraphe », coupure vers A, est intact, et A lui-même n'est jamais écrit. Il n'est pas nécessaire d'introduire une gestion de contexte arborescente dans les transclusions : l'empilement est déjà compté par la messagerie, puisque 6 pointe sur 4, qui pointe sur A.

5 Sémantique : chaque message est une règle de réécriture

Chaque message X définit une règle de réécriture positionnée <X> → X. Le membre gauche <X> est le texte que X remplace, l'intérieur qu'il recouvre ; le membre droit X est ce qu'il pose au-dessus.

La lettre X n'a pas le même sens des deux côtés de la règle. Dans <X>, X est un nom : le hash du message. Comme membre droit, X est une valeur : le texte que le message pose au-dessus, lu dans son contexte. Pour C, le membre droit est « avec un repentir non trivial. Pour continuer : », et non le hash `70fd...3c77`. Quand l'ambiguïté compte, nous notons $[\![X]\!]$ cette valeur, et la règle s'écrit <X> → $[\![X]\!]$.

Message	<X> (membre gauche)	Longueur
A	Ceci est un paragraphe.	23
B	paragraphe	10
C	réécrit	7
4	Ceci	4
5	.	1
6	paragraphe (dans la copie)	10

Table 3: Membres gauches des messages.

Une racine comme A ne remplace rien : elle marque tout le texte qu'elle pose, et <A> en est la longueur totale. Pour une réécriture, <X> est son emprise sur la couche du dessous.

Le point final de A disparaît du rendu de C, mais n'appartient pas à <C>. Sa suppression passe par la coupure <A:23>, c'est-à-dire par le contexte. Décontextualisée, cette suppression a la même relation que le message 5, « . » → « » ; seul le contexte les distingue, l'une agit sur A, l'autre sur sa copie dans 4.

5.1 L'opérateur @>

L'opérateur @> associe à un hash sa relation de réécriture décontextualisée. La notation des opérateurs est héritée des déclarations mixfixes d'ELAN [5], le langage de réécriture avec stratégies développé au LORIA. En ELAN, le symbole @ marque la place de chaque argument dans le nom de l'opérateur, par exemple `@ + @` [6] ; nous reprenons cette notation. On écrit @> pour parler de l'opérateur, et on remplit l'emplacement pour l'appliquer, comme ci-dessous ; de même, @>> s'applique en `70fd...3c77`>>.

```
70fd...3c77>réécrit->avec un repentir non trivial. Pour continuer :
```

Cette relation perd beaucoup d'entropie : elle ne dit ni où se trouvait « réécrit », ni que le point a disparu. Elle suffit pourtant à des objectifs de preuve : vérifier qu'un rendu résulte bien de la réécriture d'un fragment donné, sans divulguer l'historique.

5.2 Repentir

En peinture, le repentir est la couche antérieure qui reste visible sous la nouvelle. <B> en est l'exact équivalent : le texte que B recouvre reste désignable, et un message ultérieur peut le citer. Une réécriture ne détruit rien ; elle pose une couche.

5.3 Graphe de dérivation

Le fil de messagerie est un graphe de dérivation au sens de la réécriture de graphes. Chaque message est un pas (règle, occurrence), identifié par son hash. En réécriture classique, deux pas dont les emprises sont disjointes commutent [7]. Notre langage ajoute une contrainte : une coupure ne peut être dépensée qu'une fois (section 6). Les messages 5 et 6 partagent la coupure <4:23> : ils sont en conflit, bien que leurs emprises (24 d'un côté, 14 à 23 de l'autre) soient disjointes. C'est à ce niveau, au caractère et non à la ligne, que se situe le conflit.

6 Le journal : l'opérateur `@>>`

L'opérateur `@>>` réintègre le contexte que `@>` a perdu, à la manière d'un Journal officiel. Il fait le pont entre la chaîne de hashes et le droit.

La légistique française pratique depuis longtemps ce langage. Une modification type s'écrit : « À l'article L. 123-4, les mots : « paragraphe » sont remplacés par les mots : « texte réécrit ». » On y retrouve chaque élément du modèle :

Légistique	Notre langage
Les mots cités qui sont remplacés	Membre gauche `<X>`
Les mots qui les remplacent	Membre droit X
Article, alinéa	Coupure `<m:i>`
Journal officiel, en ajout seul	Fil de messages haché
Version consolidée	Rendu du panneau droit
Publication, opposabilité	Passage de `@>` à `@>>`

Table 4: Correspondance entre légistique et langage de messagerie.

Rédigé en légistique, `70fd...3c77>>` se lirait :

> Message 70fd...3c77. Dans le message 0631...7a3c, après le septième caractère, le mot : « réécrit » est remplacé par les mots : « avec un repentir non trivial. Pour continuer : ». Le point final du message 749a...986b est supprimé.

La dernière phrase est exactement ce que `@>` avait perdu.

Deux différences séparent les deux systèmes. Le légiste adresse par article et alinéa, une granularité proche de la ligne de `ed` ; notre langage adresse au caractère. Le légiste cite le membre gauche pour qu'on le retrouve ; le hash garantit qu'on le retrouve. Le Journal officiel certifie par l'autorité de la publication, la chaîne de hashes par le calcul.

6.1 Versions et titres exécutoires

Chaque hash est aussi une étiquette de version : il désigne le document tel qu'il est dans ce message, avec tout son historique. Un titre lie un nom à l'une de ces versions :

```
7. Titre<6
```

Le message 7 rend la ressource 6 exécutoire pour le titre, identifié dans son contexte. En droit, c'est la promulgation qui rend un texte exécutoire ; ici, c'est un message, lui-même haché et attribuable.

6.2 La règle de la double dépense

Les coupures suivent la règle de la double dépense : une coupure ne peut intervenir qu'une seule fois, comme une sortie de transaction dans une chaîne de blocs [14], ou une ressource en logique linéaire [9]. Les messages 5 et 6 utilisent tous deux `<4:23>` : ils sont en conflit. Le DAG porte alors deux versions concurrentes issues de 4, et un titre n'en rend qu'une exécutoire ; le message 7 désigne 6.

6.3 Le contrôle par les usagers

Des versions en conflit peuvent coexister dans le DAG pour un même titre. Le langage ne dit pas laquelle doit être exécutoire : il rend le choix explicite, daté et traçable, sous la forme d'un message de titre. S'ouvre ici la question du contrôle par les usagers, et de la démocratie numérique dans cet univers : qui peut émettre un message de titre, et selon quelle procédure.

7 Jonction avec XML

Les balises du langage empruntent la notation d'XML, et leur séquençage pour le hachage suit le même principe qu'une canonicalisation XML : une forme sérialisée unique, sur laquelle le hash est calculé.

La cible de la jonction est le standard OASIS Akoma Ntoso (LegalDocML) [1], qui modélise les textes législatifs en XML. Nous y projetons `@>>` : l'acte modificateur porte une disposition `<mod>` dans son corps et une métadonnée `<textualMod>` dans ses `<activeModifications>`. Le type d'une `textualMod` prend sa valeur parmi `repeal`, `substitution`, `insertion`, `replacement`, `renumbering`, `split` et `join` [3]. Ses enfants sont `source`, `destination`, puis `old` et `new` [2].

Notre langage	Akoma Ntoso
Hash du message, `70fd...3c77`	Attribut `GUID` de la `textualMod`
Message lui-même	Disposition `<mod>`, référencée par `<source>`
Base de la coupure d'ouverture, `0631...7a3c`	`<destination>`
Membre gauche `<C>`, « réécrit »	`<old>`, vers un `<quotedText>`
Membre droit C	`<new>`, vers un `<quotedText>`
Coupure de fermeture `<749a...986b:23>`	Seconde `textualMod`, de type `repeal`, de même relation décontextualisée que le message 5

Table 5: Projection de `@>>` dans Akoma Ntoso.

Appliqué à C, `70fd...3c77>>` se projette ainsi (adresses simplifiées en fragments ; Akoma Ntoso utilise des IRI) :

```
<!-- corps de l'acte modificateur -->
<mod eId="mod_70fd">Dans le message <ref href="#0631...7a3c">0631...7a3c</ref>,
  après le septième caractère, le mot :
  <quotedText eId="mod_70fd__qtext_1">réécrit</quotedText>
  est remplacé par les mots :
  <quotedText eId="mod_70fd__qtext_2">avec un repentir non trivial.
    Pour continuer :</quotedText>.
  Le point final du message <ref href="#749a...986b">749a...986b</ref>
  est supprimé.</mod>

<!-- métadonnées -->
<activeModifications>
  <textualMod type="substitution" GUID="70fd...3c77">
    <source href="#mod_70fd"/>
    <destination href="#0631...7a3c"/>
    <old href="#mod_70fd__qtext_1"/>
    <new href="#mod_70fd__qtext_2"/>
  </textualMod>
  <textualMod type="repeal">
    <source href="#mod_70fd"/>
    <destination href="#749a...986b"/>
  </textualMod>
</activeModifications>
```

La projection rend visible la différence entre les deux opérateurs. `70fd...3c77>` ne retient que la paire `<old>` / `<new>` ; `70fd...3c77>>` ajoute `<source>`, `<destination>` et la seconde modification, celle qui supprime le point, dont la relation décontextualisée est celle du message 5.

Un écart reste à combler. Akoma Ntoso adresse une destination par l'identifiant d'une partition (article, alinéa), granularité proche de la ligne ; la coupure au caractère n'y a pas d'équivalent natif. Il faudra la porter dans l'adresse de destination ou par une extension du schéma.

8 Agents et travaux liés

8.1 Agents

Pour une collection d'agents, le langage offre une édition compacte, vérifiable par le hash, et un historique où chaque contribution est attribuable. Une difficulté demeure : les modèles de langage comptent mal les caractères, à cause de la tokenisation. Un langage à offsets numériques est simple à vérifier mais fragile à produire ; il gagne à s'accompagner d'une boucle de validation, par exemple en renvoyant le rendu à l'agent avant publication.

8.2 Travaux liés

Travaux	Rapport avec notre modèle
Xanadu [15]	Transclusion adressée dans des documents immuables, pensée en 3D ; notre modèle la ramène à 2D
Transformation opérationnelle [8]	Édition concurrente au caractère, avec transformation des positions
CRDT [16]	Convergence sans coordination ; identifiants par caractère plutôt qu'offsets
Darcs, Pijul [13]	Théorie des patchs, commutation, approche catégorique
Grammaires de compression (SLP)	Un fil de messages est un SLP multi-documents, parent de LZ77 [17] ; accès aléatoire sans expansion [4, 12]
VCDIFF [11], packfiles Git	Deltas binaires de copie et d'insertion à l'octet

Table 6: Travaux liés.

Notre modèle évite les conflits au stockage, puisque tout est en ajout seul. Le conflit se déplace vers le choix de la tête affichée dans le panneau droit, et s'analyse comme une paire critique.

9 Conclusion et perspectives

Un langage de messagerie à deux constructions, la coupure et la transclusion, suffit à réécrire un document au caractère, là où la tradition issue de `ed` s'arrête à la ligne. La coupure compte pour un caractère et rend les adresses stables ; la transclusion compte pour ce qu'elle marque et rend son intérieur adressable. Le passage des numéros aux hashes ne change aucun offset : distribuer la messagerie revient à renommer ses messages.

La transclusion en deux dimensions donne un intérieur et un extérieur. On n'écrit jamais dans un message, seulement au-dessus d'une copie ; l'empilement des couches est compté par la messagerie elle-même, sans contexte arborescent. C'est le fondement de la sécurité du modèle : un texte publié est immuable, et toute modification est un nouveau message, attribuable et haché.

Lire chaque message comme une règle `<X>` $\to$ X place le modèle dans le cadre de la réécriture. Le fil est un graphe de dérivation, et le membre gauche reste désignable : ce qu'une réécriture recouvre n'est jamais perdu. Deux opérateurs en découlent. `X>` donne la relation décontextualisée : elle perd l'emplacement et les effets de bord, mais suffit à prouver qu'un fragment a été réécrit en un autre. `X>>` restitue le contexte à la manière d'un Journal officiel ; projeté dans Akoma Ntoso, il retrouve la forme d'une modification législative, avec sa source, sa destination et ses mots cités.

9.1 Applications

Le **prompt management** est une application immédiate. Un prompt de production est un document réécrit par plusieurs humains et plusieurs agents ; il se prête exactement au modèle :

- chaque variante d'un prompt est un message, identifié par son hash, et le fil en garde la lignée complète ;
- `X>` isole ce qu'une variante a changé, ce qui permet d'attribuer un écart d'évaluation à une réécriture précise ;
- `X>>` fournit la trace contextualisée qu'exigent l'audit et les obligations de traçabilité ;
- la transclusion partage des fragments communs (consignes de sécurité, persona, format de sortie) ; une variante ne peut les altérer qu'en écrivant au-dessus, de façon visible ;
- un agent émet une réécriture au caractère plutôt qu'un prompt entier, ce qui réduit le volume échangé.

D'autres applications suivent la même logique : la consolidation de textes juridiques, l'écriture collaborative entre humains et agents, et la mémoire partagée d'une collection d'agents.

9.2 Perspectives

Six chantiers restent ouverts :

1. formaliser la règle de la double dépense, et son rapport avec l'indépendance des pas et les paires critiques ;
2. combler l'écart avec Akoma Ntoso, en portant la coupure au caractère dans l'adresse de destination ou par une extension du schéma ;
3. outiller les agents, avec une validation des offsets avant publication ;
4. réaliser un prototype de la messagerie à trois panneaux et mesurer le rapport entre taille brute et taille rendue ;
5. préciser la politique de sécurité de l'intérieur : qui peut écrire au-dessus de quelle copie ;
6. définir le contrôle par les usagers des messages de titre, c'est-à-dire la gouvernance démocratique des versions exécutoires.

Références

[1] OASIS. *Akoma Ntoso Version 1.0*. OASIS Standard, LegalDocML Technical Committee, 2018.

[2] OASIS. Akoma Ntoso, élément `textualMod`. `https://docs.oasis-open.org/legaldocml/akn-core/v1.0/os/part2-specs/os-part2-specs_xsd_Element_textualMod.html`

[3] OASIS. Akoma Ntoso, type `TextualMods`. `https://docs.oasis-open.org/legaldocml/akn-core/v1.0/os/part2-specs/os-part2-specs_xsd_Simple_Type_TextualMods.html`

[4] P. Bille, G. M. Landau, R. Raman, K. Sadakane, S. R. Satti, O. Weimann. Random access to grammar-compressed strings. *SODA*, 2011.

[5] P. Borovanský, C. Kirchner, H. Kirchner, P.-E. Moreau. ELAN from a rewriting logic point of view. *Theoretical Computer Science*, 285(2):155–185, 2002.

[6] P. Borovanský, H. Cirstea, E. Deplagne, H. Dubois, C. Kirchner, H. Kirchner, P.-E. Moreau, Q.-H. Nguyen, C. Ringeissen, M. Vittek. *ELAN User Manual* (ELAN V3.7). LORIA, Nancy, 2006.

[7] H. Ehrig, K. Ehrig, U. Prange, G. Taentzer. *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.

[8] C. A. Ellis, S. J. Gibbs. Concurrency control in groupware systems. *ACM SIGMOD*, 1989.

[9] J.-Y. Girard. Linear logic. *Theoretical Computer Science*, 50(1), 1987.

[10] J. W. Hunt, M. D. McIlroy. An algorithm for differential file comparison. Computing Science Technical Report 41, Bell Laboratories, 1976.

[11] D. Korn, J. MacDonald, J. Mogul, K. Vo. The VCDIFF generic differencing and compression data format. RFC 3284, 2002.

[12] M. Lohrey. Algorithmics on SLP-compressed strings: a survey. *Groups Complexity Cryptology*, 4(2), 2012.

[13] S. Mimram, C. Di Giusto. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science*, 298, 2013.

[14] S. Nakamoto. Bitcoin: a peer-to-peer electronic cash system. 2008.

[15] T. H. Nelson. *Literary Machines*. Mindful Press, 1981.

[16] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-free replicated data types. *SSS*, 2011.

[17] J. Ziv, A. Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3), 1977.